Prénom ..... Date .....

Complexité algorithmique

Exercice 1 : Somme des n premiers nombres

1) Annoter le programme pour compter combien de fois chaque ligne est exécutée.

#	Programme	Opérations	Executions	Sous-total
1	`somme = 0`			
2	`for i in range(n):`			
3	`    somme = somme + (i + 1)`			
4	`print(somme)`			

2) Trouver la fonction qui lie le nombre d'opérations à n

.....

3) Déterminer la classe de complexité du programme

.....

Exercice 2 : Somme des n premiers nombres – forme fermée

Il y a une manière plus rapide de calculer la somme des n premiers nombres, grâce à cette identité :

$$1 + 2 + 3 + \cdots + (n-1) + n = \frac{n \times (n+1)}{2}$$

1) Annoter le programme pour compter combien de fois chaque ligne est exécutée

#	Programme	Opérations	Executions	Sous-total
1	`somme = (n * (n + 1)) / 2`			
2	`print(somme)`			

2) Trouver la fonction qui lie le nombre d'opérations à n

.....

3) Déterminer la classe de complexité du programme

.....

Exercice 3 : Table de multiplication

Le programme suivant affiche la table de multiplication de 1 à n.

1) Annoter le programme pour compter combien de fois chaque ligne est exécutée

#	Programme	Opérations	Executions	Sous-total
1	`for i in range(n):`			
2	`    for j in range(n):`			
3	`        print((i+1) * (j+1))`			

2) Trouver la fonction qui lie le nombre d'opérations à n

.....

3) Déterminer la classe de complexité du programme

.....

Exercice 4 : Déterminer si un nombre est premier

1) Annoter le programme pour compter combien de fois chaque ligne est exécutée

#	Programme	Opérations	Executions	Sous-total
1	`for i in range(2, n):`			
2	`  if n % i == 0:`			
3	`    print(i, "divise", n)`			
4	`  else:`			
5	`    print(".")`			

Rappel : `n % i` calcule le reste de la division entière de `n` par `i`.

2) Trouver la fonction qui lie le nombre d'opérations à n

.....

3) Déterminer la classe de complexité du programme

.....